PAAVAI ENGINEERING COLLEGE

(AUTONOMOUS)

(An Autonomous Institution Approved by AICTE & Affiliated to Anna University)

(NAAC (UGC) With 'A' Grade)

Paavai Nagar, NH-44, Pachal, Namakkal(DT) - 637 018.

Name ..... Branch .....

Register No. ..... Roll No. .....

Certified that this is the Bonafide Record Work done by the above Student of the .....

Laboratory during the year 20 - 20

Signature of lab In - Charge

Signature of Head of the Dept.

Submitted for the Practical Examination held on .....

Internal Examiner

External Examiner

Ex.No:	**Implementation of Python basic libraries (Statistics, Math, NumPy, SciPy) including the use of mathematical functions, array attributes and operations, matrix transformations, and determinant computation**
Date:	

Aim

To implement Python basic libraries such as **Math, NumPy, SciPy and Statistics** and demonstrate mathematical functions, array attributes, matrix operations, determinant calculation, and eigenvalues.

Algorithm

Step 1 : Start the program.

Step 2 : Import required Python libraries such as **math, numpy, scipy, and statistics**.

Step 3 : Implement mathematical functions using the **math library**.

Step 4 : Create arrays and perform operations using **NumPy library**.

Step 5 : Reshape arrays into different dimensions.

Step 6 : Generate matrices using NumPy.

Step 7 : Compute determinant using **SciPy library**.

Step 8 : Compute eigenvalues and eigenvectors using **SciPy**.

Step 9 : Display the results.

Step 10 : Stop the program.

Program 1

Mathematical Functions using Math Library

```
import math

x = 10.7
print("Floor value:", math.floor(x))
print("Ceil value:", math.ceil(x))
print("Square root:", math.sqrt(25))
print("Integer square root:", math.isqrt(16))
print("GCD:", math.gcd(20,30))
```

Output

```
Floor value: 10
Ceil value: 11
```

Square root: 5.0
Integer square root: 4
GCD: 10

Program 2

NumPy Array Attributes

```
import numpy as np
a = np.array([[1,2,3],[4,5,6]])
print("Array:\n",a)
print("Dimension:",a.ndim)
print("Shape:",a.shape)
print("Size:",a.size)
```

Output

```
Array:
[[1 2 3]
 [4 5 6]]
Dimension: 2
Shape: (2,3)
Size: 6
```

Program 3

NumPy Array Operations

```
import numpy as np
a = np.array([10,20,30,40])
print("Sum:",a.sum())
print("Mean:",a.mean())
print("Sorted array:",np.sort(a))
print("Sin values:",np.sin(a))
```

Output

```
Sum: 100
Mean: 25.0
Sorted array: [10 20 30 40]
Sin values: [-0.54402111  0.91294525 -0.98803162  0.74511316]
```

Program 4

Reshape 1D List into 2D and 3D Matrix

```
import numpy as np
a = np.array([1,2,3,4,5,6,7,8])
b = a.reshape(2,4)
c = a.reshape(2,2,2)
print("2D Matrix:\n",b)
print("3D Matrix:\n",c)
```

Output

```
2D Matrix:
 [[1 2 3 4]
 [5 6 7 8]]
3D Matrix:
 [[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]
```

Program 5

Generate Random Matrix

```
import numpy as np
a = np.random.randint(1,10,(3,3))
print("Random Matrix:\n",a)
```

Output

```
Random Matrix:
 [[9 8 5]
 [9 1 6]
 [4 3 9]]
```

Program 6

Determinant of Matrix using SciPy

```
import numpy as np

A = np.array([[2,3],[1,4]])

det = np.linalg.det(A)

print("Matrix:\n", A)
print("Determinant:", det)
```

Output

```
Matrix:
 [[2 3]
 [1 4]]
Determinant: 5.000000000000001
```

Program 7

Eigenvalues and Eigenvectors using SciPy

```
import numpy as np
A = np.array([[1, 2], [3, 4]])
value, vector = np.linalg.eig(A)
print("Eigen Values:", value)
print("Eigen Vectors:\n", vector)
```

Output

```
Eigen Values: [-0.37228132  5.37228132]
Eigen Vectors:
 [[-0.82456484 -0.41597356]
 [ 0.56576746 -0.90937671]]
```

.

PAAVAI ENGINEERING COLLEGE (Autonomous)		
DESCRIPTION	**MAX. MARKS**	**MARKS AWARDED**
Preparation& Conduction	**20**	
Observation &Results	**10**	
Record Completion	**05**	
Viva Voce	**05**	
TOTAL	**40**	

Result:

Thus, the implementation of **Python basic libraries such as Math, NumPy, SciPy and Statistics** and their operations were successfully executed.

Ex.No:2	Implementation of Python Libraries for ML. applications such as Pandas and Matplotlib. to Create a Series using pandas and display,Access the index and the values of our Series, compare an array using NumPy with a series using pandas, Load datasets in a Data frame variable using pandas and Usage of different methods in Matplotlib.
Date:	

Aim:

To implement Python libraries for Machine Learning applications such as **Pandas and Matplotlib** to create and manipulate Series objects, compare NumPy arrays with Pandas Series, load datasets into DataFrames, and visualize data using different plotting methods.

Algorithm:

Step 1: Start the program

Step 2: Import the required libraries such as **pandas, numpy, and matplotlib**.

Step 3: Create a **Series object** using pandas and display it.

Step 4: Access the **index and values** of the Series.

Step 5: Compare a **NumPy array with a pandas Series**.

Step 6: Load a dataset into a **DataFrame using pandas**.

Step 7: Use **Matplotlib functions** to plot graphs.

Step 8: Display the results.

Step 9: Stop the program

Program 1

Create a Series using Pandas and Display

```
import pandas as pd
data = [10, 20, 30, 40, 50]
s = pd.Series(data)
print("Series:\n", s)
```

Output

```
Series:
0    10
1    20
2    30
3    40
4    50
dtype: int64
```

Program 2

Access Index and Values of a Series

```
import pandas as pd
data = [5, 10, 15, 20]
s = pd.Series(data)
print("Series:\n", s)
print("Index:", s.index)
print("Values:", s.values)
```

Output

```
Series:
0     5
1    10
2    15
3    20
dtype: int64
Index: RangeIndex(start=0, stop=4, step=1)
Values: [ 5 10 15 20]
```

Program 3

Compare NumPy Array with Pandas Series

```
import numpy as np
import pandas as pd
arr = np.array([1,2,3,4])
series = pd.Series([1,2,3,4])
print("NumPy Array:", arr)
print("Pandas Series:\n", series)
```

Output

```
NumPy Array: [1 2 3 4]
Pandas Series:
0    1
1    2
2    3
3    4
dtype: int64
```

Program 4

Load Dataset into DataFrame using Pandas

```
import pandas as pd
data = {
    "Name": ["A", "B", "C"],
    "Age": [21, 22, 23],
    "Marks": [85, 90, 88]
}

df = pd.DataFrame(data)

print("DataFrame:\n", df)
```

Output

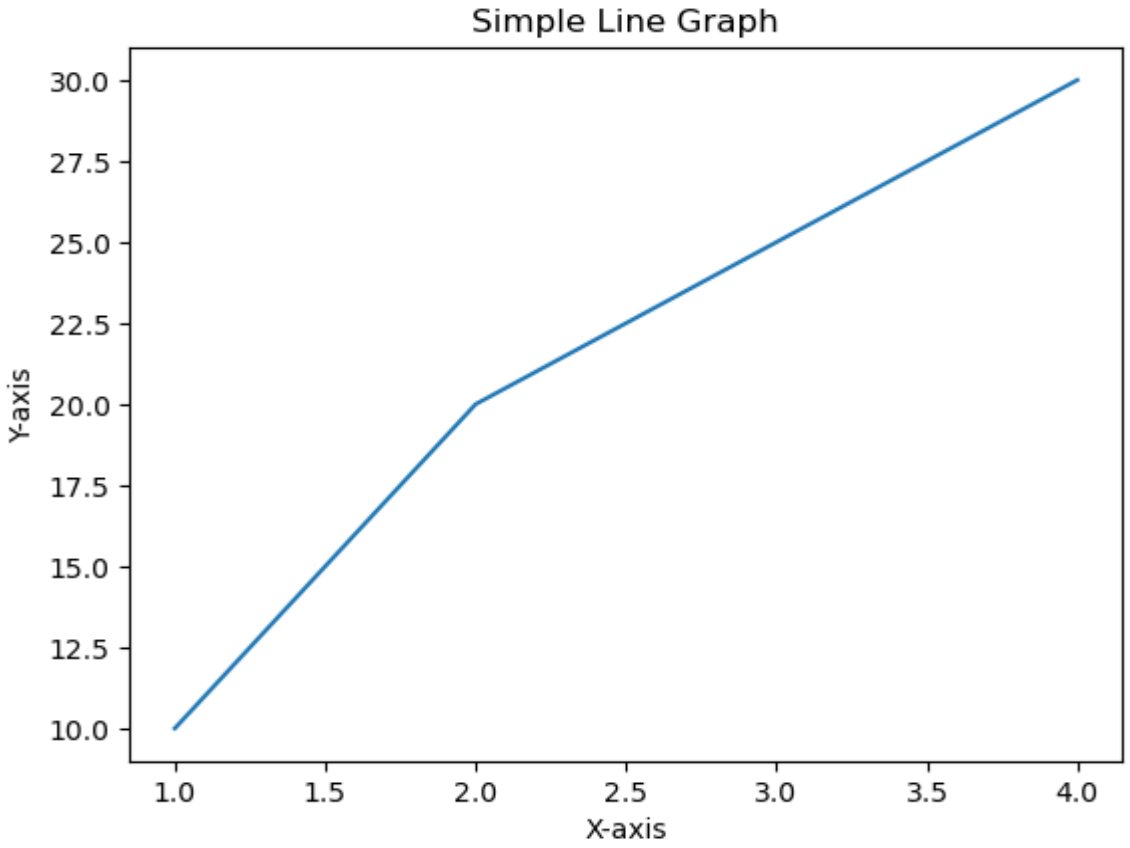
```
DataFrame:
  Name  Age  Marks
0    A   21     85
1    B   22     90
2    C   23     88
```

Program 5

Plot Graph using Matplotlib

```
import matplotlib.pyplot as plt
x = [1,2,3,4]
y = [10,20,25,30]
plt.plot(x,y)
plt.title("Simple Line Graph")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

output


Simple Line Graph
30.0
27.5
25.0
22.5
20.0
17.5
15.0
12.5
10.0
Y-axis
1.0
1.5
2.0
2.5
3.0
3.5
4.0
X-axis

PAAVAI ENGINEERING COLLEGE (Autonomous)		
DESCRIPTION	**MAX. MARKS**	**MARKS AWARDED**
Preparation& Conduction	**20**	
Observation &Results	**10**	
Record Completion	**05**	
Viva Voce	**05**	
TOTAL	**40**	

RESULT:

Thus, the implementation of **Python libraries for Machine Learning applications such as Pandas and Matplotlib** was successfully performed

Ex.No:5	WRITE A PYTHON PROGRAM TO IMPLEMENT SIMPLE LINEAR REGRESSION AND PLOT THE GRAPH
Date:	

Aim:

To implement **Simple Linear Regression using Python** and plot the relationship between the independent variable and dependent variable using a graph.

Algorithm

Step 1: Start the program

Step 2: Import the required libraries such as **numpy, matplotlib, and sklearn**.

Step 3: Define the independent variable (X) and dependent variable (Y).

Step 4: Create the **Linear Regression model**.

Step 5: Train the model using the given data.

Step 6: Predict the output values using the model.

Step 7: Plot the regression line and data points using **Matplotlib**.

Step 8: Display the graph.

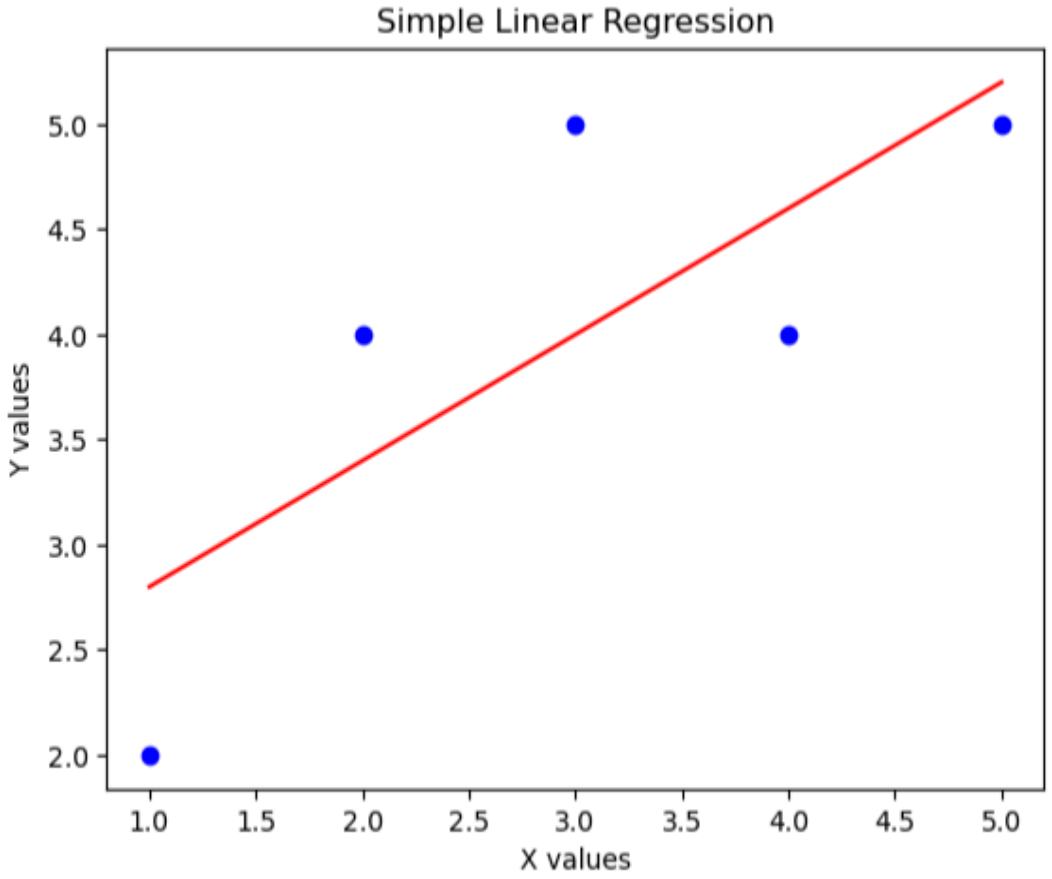
Step 9: Stop the program

Program

```
import numpy as np
import matplotlib.pyplot as plt

# Dataset
x = np.array([1,2,3,4,5])
y = np.array([2,4,5,4,5])
# Calculate mean
mean_x = np.mean(x)
mean_y = np.mean(y)
# Calculate slope (m)
```

```
m = np.sum((x-mean_x)*(y-mean_y)) / np.sum((x-mean_x)**2)
# Calculate intercept (b)
b = mean_y - m*mean_x

# Regression line
y_pred = m*x + b
# Plot original data
plt.scatter(x,y,color='blue')
# Plot regression line
plt.plot(x,y_pred,color='red')
plt.title("Simple Linear Regression")
plt.xlabel("X values")
plt.ylabel("Y values")
plt.show()
print("Slope:",m)
print("Intercept:",b)
```

output




```
Slope: 0.6
Intercept: 2.2
```

PAAVAI ENGINEERING COLLEGE (Autonomous)		
DESCRIPTION	**MAX. MARKS**	**MARKS AWARDED**
Preparation& Conduction	**20**	
Observation &Results	**10**	
Record Completion	**05**	
Viva Voce	**05**	
TOTAL	**40**	

RESULT:

Thus, the **Simple Linear Regression model was implemented successfully and the regression graph was plotted using Python.**

Ex.No:7	Write a Python program to implement K-Means Clustering Algorithm for different K values
Date:	

Aim:

To implement the **K-Means Clustering Algorithm using Python** and analyze clustering results for **different K values**.

Algorithm

Step 1: Start the program

Step 2: Import the required libraries such as **NumPy and Matplotlib**.

Step 3: Create or load the dataset.

Step 4: Select the number of clusters **K**.

Step 5: Initialize **K centroids randomly** from the dataset.

Step 6: Calculate the **distance between each data point and centroid**.

Step 7: Assign each data point to the **nearest centroid** to form clusters.

Step 8: Update the centroid by calculating the **mean of all points in the cluster**.

Step 9: Repeat steps **6–8 until centroids do not change**.

Step 10: Display the clusters and centroids using a graph.

Step 11: Repeat the process for **different K values**.

Step 12: Stop the program

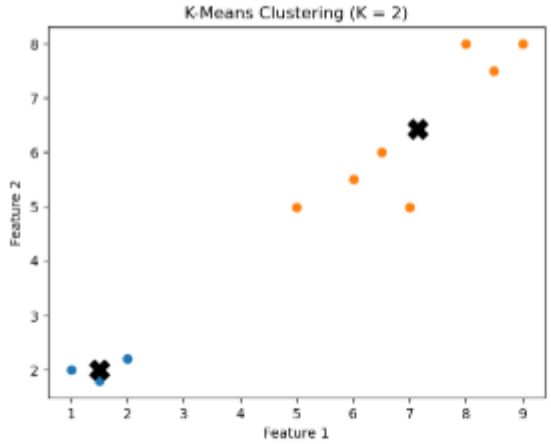
Program

```
import numpy as np
import matplotlib.pyplot as plt

# Sample dataset
X = np.array([
    [1,2],[1.5,1.8],[2,2.2],
    [8,8],[8.5,7.5],[9,8],
    [5,5],[6,5.5],[6.5,6],[7,5]
])

# K-Means function
def kmeans(X, k, max_iter=100):

    # Initialize centroids randomly
    np.random.seed(0)
    centroids = X[np.random.choice(len(X), k, replace=False)]

    for _ in range(max_iter):

        clusters = [[] for _ in range(k)]

        # Assign points to nearest centroid
        for point in X:
            distances = [np.linalg.norm(point - centroid) for centroid in centroids]
            cluster_index = np.argmin(distances)
            clusters[cluster_index].append(point)

        # Update centroids
        new_centroids = []
        for cluster in clusters:
            new_centroids.append(np.mean(cluster, axis=0))

        new_centroids = np.array(new_centroids)

        # Check convergence
        if np.all(centroids == new_centroids):
            break

        centroids = new_centroids

    return clusters, centroids


# Different K values
```

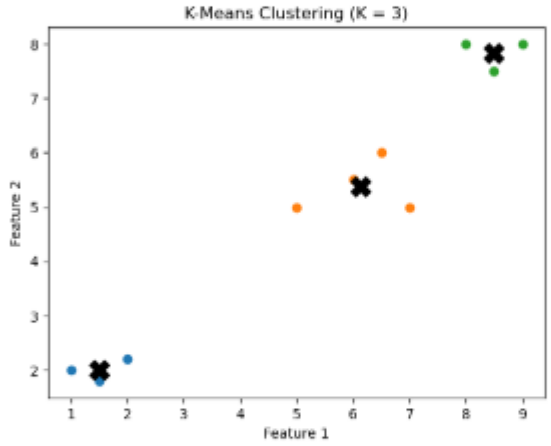
```
K_values = [2,3,4]

for k in K_values:

    clusters, centroids = kmeans(X, k)

    print("K =", k)
    print("Centroids:\n", centroids)

    # Plot clusters
    plt.figure()

    for cluster in clusters:
        cluster = np.array(cluster)
        plt.scatter(cluster[:,0], cluster[:,1])

    plt.scatter(centroids[:,0], centroids[:,1], color='black', marker='X', s=200)

    plt.title("K-Means Clustering (K = {})".format(k))
    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")
    plt.show()
```

output

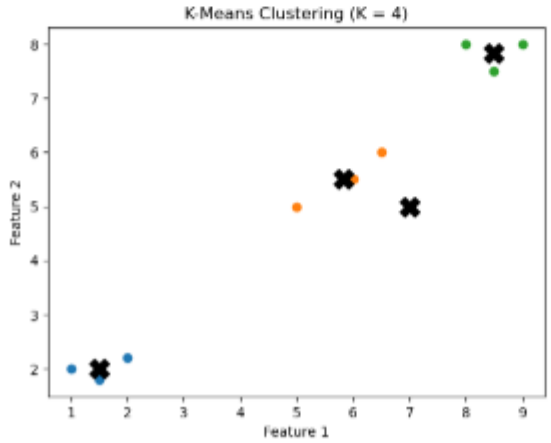
```
K = 2
Centroids:
 [[1.5        2.        ]
 [7.14285714 6.42857143]]
```




```
K = 3
Centroids:
 [[1.5        2.        ]
 [6.125      5.375     ]
 [8.5        7.83333333]]
```




```
K = 4
Centroids:
 [[1.5        2.        ]
 [5.83333333 5.5       ]
 [8.5        7.83333333]
 [7.         5.        ]]
```



PAAVAI ENGINEERING COLLEGE (Autonomous)		
DESCRIPTION	**MAX. MARKS**	**MARKS AWARDED**
Preparation& Conduction	**20**	
Observation &Results	**10**	
Record Completion	**05**	
Viva Voce	**05**	
TOTAL	**40**	

RESULT:

Thus, the K-Means Clustering Algorithm was successfully implemented using Python and clustering results were obtained for different K values.

Ex.No:10	**Write a Python program to implement Intrusion Detection System using Support Vector Machine (SVM)**
Date:	

Aim:

To develop an Intrusion Detection System (IDS) **using the** Support Vector Machine (SVM) algorithm in Python **to classify network activities as normal or intrusion.**

Algorithm

Step 1: Start the program

Step 2: Import the required libraries such as **NumPy and Matplotlib**.

Step 3: Prepare a dataset containing **network traffic features and labels** (normal or attack).

Step 4: Separate the dataset into **input features (X)** and **target labels (Y)**.

Step 5: Initialize the **SVM classifier**.

Step 6: Train the SVM model using the training dataset.

Step 7: Use the trained model to **predict the class of test data**.

Step 8: Compare predicted values with actual values.

Step 9: Calculate the **accuracy of the model**.

Step 10: Display the prediction results.

Step 11: Stop the program

Program

```
import numpy as np

# Sample dataset
# Features: [packet_size, login_attempts]
```

```
X = np.array([
    [100,1],
    [200,2],
    [150,1],
    [300,4],
    [350,5],
    [400,6]
])

# Labels: 0 = Normal, 1 = Intrusion
Y = np.array([0,0,0,1,1,1])

# Simple SVM implementation
def train_svm(X, Y, lr=0.001, lambda_param=0.01, n_iters=1000):

    n_samples, n_features = X.shape
    w = np.zeros(n_features)
    b = 0

    for _ in range(n_iters):
        for idx, x_i in enumerate(X):
            condition = Y[idx] * (np.dot(x_i, w) - b) >= 1

            if condition:
                w -= lr * (2 * lambda_param * w)
            else:
                w -= lr * (2 * lambda_param * w - np.dot(x_i, Y[idx]))
                b -= lr * Y[idx]

    return w, b

# Convert labels to -1 and 1
Y_svm = np.where(Y == 0, -1, 1)

# Train model
w, b = train_svm(X, Y_svm)

# Prediction function
def predict(X, w, b):
    approx = np.dot(X, w) - b
    return np.sign(approx)

# Test prediction
predictions = predict(X, w, b)

print("Predictions:", predictions)
```

Output

Predictions: [-1 -1 -1 1 1 1]

PAAVAI ENGINEERING COLLEGE (Autonomous)		
DESCRIPTION	**MAX. MARKS**	**MARKS AWARDED**
Preparation& Conduction	**20**	
Observation &Results	**10**	
Record Completion	**05**	
Viva Voce	**05**	
TOTAL	**40**	

RESULT:

Thus, the **Intrusion Detection System using Support Vector Machine (SVM) was successfully implemented in Python and network traffic was classified as normal or intrusion**